Reflection — Boréal Marché Customer Intelligence Platform

Olga Morgunova · 420-977-VA AI Development Project (Capstone 1) · D7 Learning Integration Assessment · release `v1.0` · Instructor: Steve Ataky, Ph.D.

1. The hardest technical problem: a deployment with no key to steal

The rule I set myself in D4 was that no AWS credential would exist in GitHub: a merge to `master` had to authenticate through GitHub's OIDC token, assume one narrowly scoped role, push an image tagged with the commit SHA and redeploy App Runner. Every piece looks simple in the documentation. Together they failed three times, and each failure looked the same from the outside: `AccessDenied`, nothing more.

The temptation was to widen the policy until it passed. Doing that would have produced a pipeline that works and proves nothing, so I made a rule of reading the evidence first. **First failure:** the role refused the token. CloudTrail's `AssumeRoleWithWebIdentity` event showed the subject GitHub actually sends. It does not carry `repo:owner/repo:…`, as I had written it, but the *immutable* owner and repository IDs. I changed the trust policy to that form. The side effect turned out to be a security gain: a repository deleted and recreated under the same name can no longer deploy. **Second failure:** `UpdateService` was denied `iam:PassRole`. My `iam:PassedToService` condition did not match what App Runner sends for its ECR access role. I removed the condition and kept `PassRole` limited to that single role ARN. **Third failure:** a re-run broke on the immutable image tag, so the push step now skips an image that already exists.

The same deployment raised two problems I could not engineer away; I could only state them. App Runner's proxy rejected Streamlit's WebSocket upgrade (a `403` from `envoy` that never reached my container), so the interface moved to Streamlit Community Cloud. And App Runner is not offered in `ca-central-1`. Serving therefore runs in `us-east-1`, which for real customer data would be a communication outside Québec under Law 25 s. 17. I documented both as deviations, with the fix (ECS Fargate in `ca-central-1`, about $30 a month more), instead of hiding them.

What I learned is a method more than a fact: on a permission error, find the record of what the system actually received before changing anything. The pipeline that resulted runs CI, CD with OIDC and a smoke test that fails unless `/version` equals the merged commit and the golden prediction is reproduced. It is the part of the project I trust most.

2. One thing I got wrong and later corrected: a monitor that was always red

In D4 I built drift monitoring with the pooled training data as the reference distribution. It seemed the obvious, conservative choice: compare live traffic with everything the model learned from. Then I replayed perfectly healthy held-out traffic through it, and the dashboard was red. The reason was in my own feature design. The training set pools four quarterly snapshots, and later snapshots have longer customer histories *by construction*, so `tenure_days` and the lifetime counts drift upward with the calendar, not with any change in customers. Against that reference, any recent traffic looks shifted.

A monitor that is always red is worse than none, because people learn to ignore it. I changed the reference to the calibration snapshot, the one closest in time to serving. I excluded `season_q` (constant within a quarter by design) and documented the residual shift that is known and expected. Healthy traffic now reads green or amber. The deliberately shifted Maritimes cohort turns 53% of features red, drops the rule fire rate from 92% to 53% and raises 18 alerts. The broader lesson changed how I test: a check must be shown to stay quiet on good input as well as to fire on bad input. I applied it again when the leakage test had to *prove* it could catch a leak (PR-AUC 0.8070 → 1.0000 with label-window features).

It was not the only correction. A value horizon of one quarter made "contact nobody" optimal. The duplicate key included the product description. `mean_gap_days` was computed between invoices rather than order days, and `overdue_ratio` reached 253,881. A promotion check compared against the registry alias instead of the served artifact. Each was found by a test or a measurement, not by rereading code. Several were in code produced with an AI assistant, which made reviewing every line against a measurement a working habit rather than a formality.

3. What I would do differently

Start from the decision, not the model. I spent early effort tuning. Tuning added less than 0.005 PR-AUC, and the 180-day spend and frequency features carry the model. The threshold decided far more: a single value-blind cut-off is what proposes only 46% of lapsing top spenders against 89% overall. I would write the campaign economics and the fairness audit in week 3, not week 14.

Build the measurement into the first release. Every dollar figure rests on a 20% save rate nobody has measured; at 10% no campaign pays. In v1.0 I finally served the randomised 10% hold-out. Its power calculation says one quarter of this dataset's size detects a save rate of about 0.16–0.21, so one to two quarters are needed. Had it run since D4, I would be reporting a measured number instead of an assumption.

Check the platform's constraints before choosing it. Region availability, WebSocket support and cloud quotas each cost me time after the choice was made. The SageMaker spot quota I requested on 2026-09-29 is still not granted (CASE_OPENED, case 179073448000753), so no managed training run exists. Local Optuna was the right primary path anyway: tuning took 1.6 minutes on 12,277 rows. But I would have filed the request in week 1 and named the fallback in the charter.

Protect the test set harder. It was scored 3 times, each disclosed and each after the choices were fixed. The second look repeated the first after a data fix. One look and a written reason for any other is the discipline I would keep.

4. What the last step taught me

Building the high-value review list for v1.0 produced the most honest result of the project. It works as designed: if the reviewer approved the whole list, top-quartile lapser recall would rise from 0.46 to 0.89 on validation. It also costs money: approving it all would yield −$8,221 under the stated assumptions, because most of those customers would have returned anyway. Fairness here is not free and the model cannot settle it. The system's job is to put the case in front of a person with the numbers, and to measure what the coupon really does. I now think that is what "responsible AI" means in a project like this one.